

Práctica #2 Github

Asignatura

Administración de Bases De Datos

Nombre del profesor/a

Dr. Richard Avilés López

Máster o Grado

Ingeniería Informática

Realizado por:

*Xavier Alejandro Gallo Rodas (Autor), Vicente Losana Moreno
(Revisor)*



Índice de contenidos

1. Introducción y objetivos:	3
2. Verificaciones previas:	3
Parte A crear Issue (Autor)	4
Parte B: Proteger la Rama main (Autor)	4
Parte C: Crear y vincular la rama (Autor)	5
Parte D: Realizar el trabajo y commits (Autor)	5
PARTE E: Push y apertura del Pull Request (Autor)	6
PARTE F: Code Review - Solicitud de cambios (Reviewer)	7
PARTE G: Responder a la revisión (Autor)	8
PARTE H Comprobar que la protección funciona	11
PARTE I Fusionar el pull request:	11
PARTE J Limpieza posterior al merge:	11
3. PREGUNTAS DE LA PRÁCTICA:	12

1. Introducción y objetivos:

En el Laboratorio 1 trabajaste en solitario. Hoy el repositorio se convierte en lo que realmente es en una empresa: un espacio compartido con reglas. Al finalizar este laboratorio, el estudiante será capaz de:

- Redactar un Issue claro y accionable en GitHub, con descripción, criterios de aceptación y etiquetas.
- Entender la diferencia entre Issue, branch, commit y Pull Request.
- Configurar una branch protegida (main) que impida el push directo y exija Pull Request, review y checks.
- Crear una branch vinculada a un Issue siguiendo una convención de nombres coherente.
- Escribir commits que referencien el Issue que resuelven.
- Abrir un Pull Request completo: título, descripción estructurada, checklist de pruebas.
- Actuar como reviewer: dar feedback constructivo, solicitar cambios y aprobar con criterio.
- Actuar como autor: responder a comentarios de revisión con nuevos commits sin perder el hilo del PR.
- Distinguir Merge commit, Squash and merge y Rebase and merge, y justificar cuál usar en cada caso.
- Cerrar un Issue automáticamente desde un commit o un PR (Closes #N).
- Limpiar el repositorio tras el merge (borrar branch, actualizar main local).

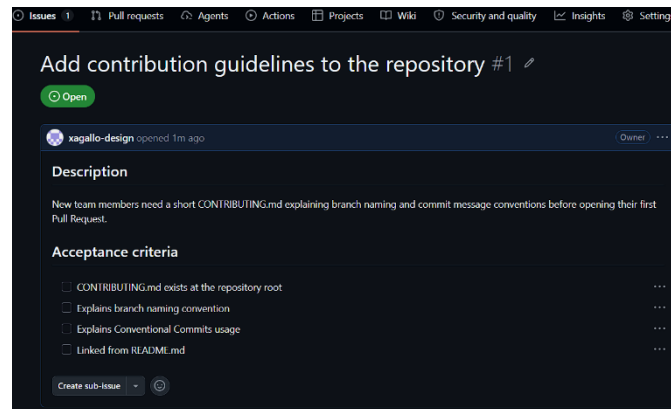
2. Verificaciones previas:

```
$ git switch main
git pull
git status
git remote -v
git branch --show-current
Already on 'main'
Your branch is up to date with 'origin/main'.
Already up to date.
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
origin  https://github.com/xagallo-design/oracle-database-lab.git (fetch)
origin  https://github.com/xagallo-design/oracle-database-lab.git (push)
main
```

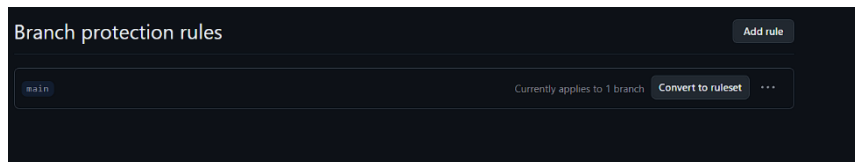
Se comprueba

Parte A crear Issue (Autor)

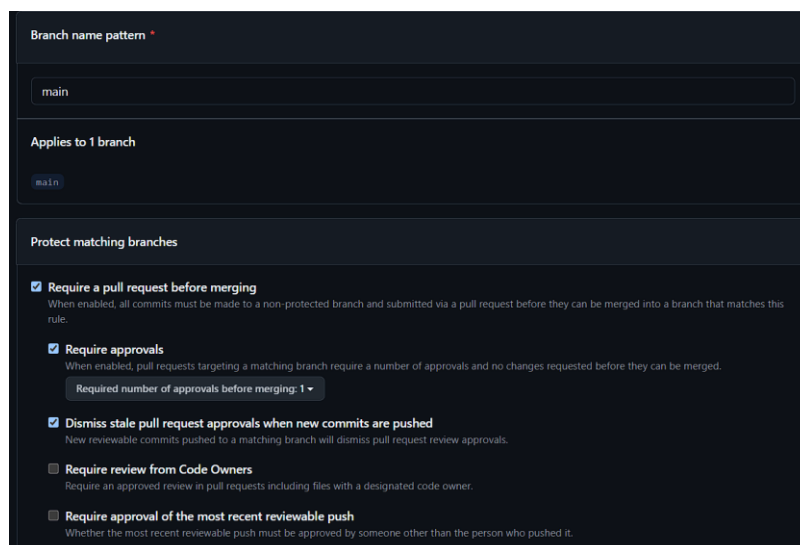


Creamos el issue con lo mencionado.

Parte B: Proteger la Rama main (Autor)



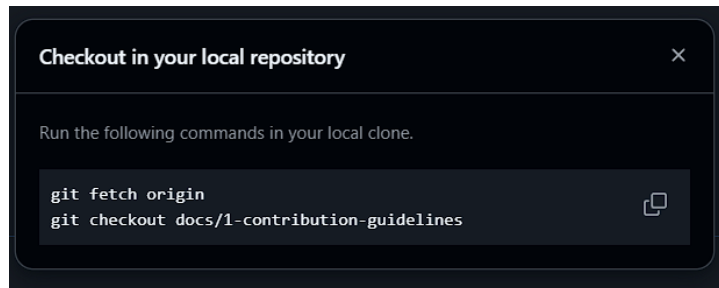
Se crea la regla de la Branch según las instrucciones.



OJO: GitHub soporta precisamente este tipo de reglas de protección: exigir Pull Request, aprobaciones, checks de estado y restricciones de push antes de permitir un merge

Parte C: Crear y vincular la rama (Autor)

La creamos desde github:



Y luego en la terminal:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (main)
$ git fetch origin
git switch docs/1-contribution-guidelines
git branch --show-current
From https://github.com/xagallo-design/oracle-database-lab
* [new branch]      docs/1-contribution-guidelines -> origin/docs/1-contribution-guidelines
branch 'docs/1-contribution-guidelines' set up to track 'origin/docs/1-contribution-guidelines'.
Switched to a new branch 'docs/1-contribution-guidelines'
docs/1-contribution-guidelines

xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$
```

OJO: Incluir el número del Issue en el nombre de la branch (1-contributing-guidelines) no es obligatorio en todas las organizaciones, ni es un estándar, pero es una práctica muy extendida porque permite rastrear de un vistazo qué branch resuelve qué Issue, incluso antes de abrir el Pull Request

Parte D: Realizar el trabajo y commits (Autor)

Creamos el archivo:

```
# Contributing to oracle-database-lab

## Branch naming
<type>/<issue number>-<short-description>, e.g. feature/56-customer-search, fix/85-customer-email-validation

## Commit messages
This project follows Conventional Commits:
<type>(scope): <description>
Common types: feat, fix, docs, test, refactor, perf, build, ci, chore

## Pull Requests
Every change must go through a Pull Request with at least one approval before merging into main.
```

Modificamos el Read me:

```
README.md > ** # Oracle Database Lab (Training Edition — Academic Version)
1 # Oracle Database Lab (Training Edition — Academic Version)
2 Training repository for Oracle Database administration,
3 testing, change management and Git workflows.
4 Línea añadida desde el navegador
5 Name: XAVIER ALEJANDRO GALLO RODAS
6 Professor: Richard Aviles Lopez
7 See [CONTRIBUTING.md](CONTRIBUTING.md) for branch and commit conventions.
```

commit y Comprobación local:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$ git add CONTRIBUTING.md README.md
$ git commit -m "docs: add contribution guidelines"

Refs #1"
warning: in the working copy of 'CONTRIBUTING.md', LF will be replaced by CRLF the next time Git touches it
[docs/1-contribution-guidelines fdff989] docs: add contribution guidelines
 2 files changed, 13 insertions(+)
 create mode 100644 CONTRIBUTING.md

xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$ git status
On branch docs/1-contribution-guidelines
Your branch is ahead of 'origin/docs/1-contribution-guidelines' by 1 commit.
  (use "git push" to publish your local commits)

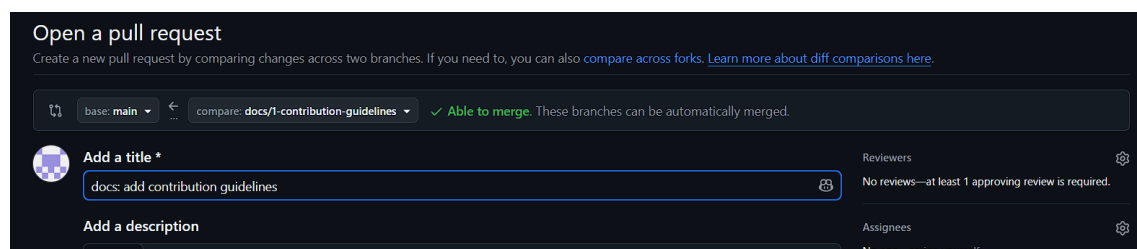
nothing to commit, working tree clean
```

PARTE E: Push y apertura del Pull Request (Autor)

Subimos la rama al repositorio remoto:

```
xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$ git push -u origin docs/1-contribution-guidelines
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 801 bytes | 801.00 KiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/xagallo-design/oracle-database-lab.git
   3fbc130..fdff989 docs/1-contribution-guidelines -> docs/1-contribution-guidelines
branch 'docs/1-contribution-guidelines' set up to track 'origin/docs/1-contribution-guidelines'.
```

En github vamos a compare and pull request:



Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also compare across forks. [Learn more about diff comparisons here.](#)

base: main ← compare: docs/1-contribution-guidelines ✓ Able to merge. These branches can be automatically merged.

Add a title *

docs: add contribution guidelines

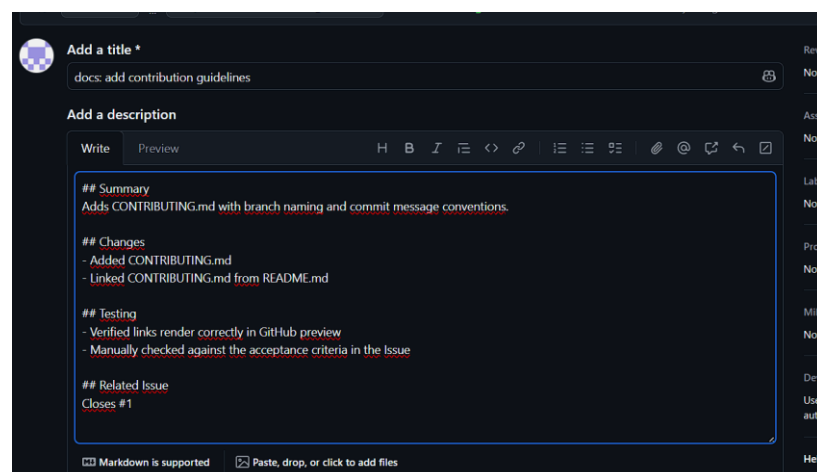
Add a description

Reviewers: No reviews—at least 1 approving review is required.

Assignees: No one—assign yourself.

Nos aseguramos de que las ramas sean: base: main ← compare: docs/1-contribution-guidelines

Configuramos el Pr:



Add a title *

docs: add contribution guidelines

Add a description

Write Preview

Summary
Adds CONTRIBUTING.md with branch naming and commit message conventions.

Changes

- Added CONTRIBUTING.md
- Linked CONTRIBUTING.md from README.md

Testing

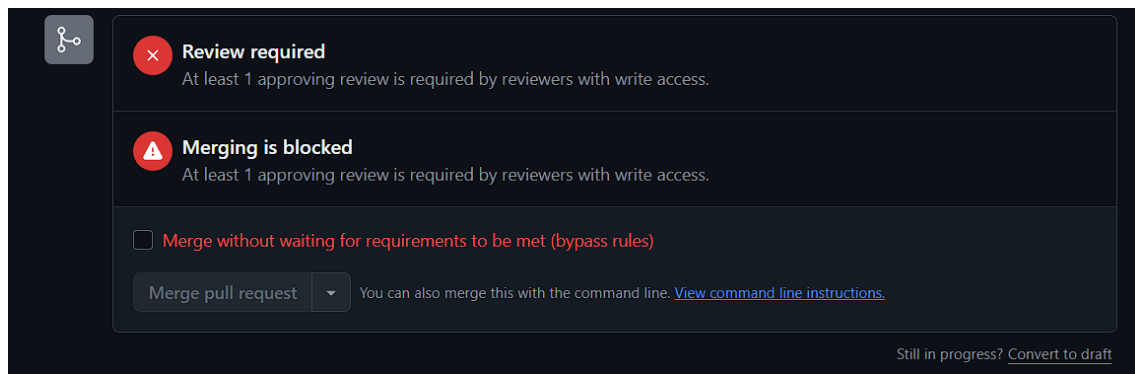
- Verified links render correctly in GitHub preview
- Manually checked against the acceptance criteria in the Issue

Related Issue

Closes #1

Markdown is supported Paste, drop, or click to add files

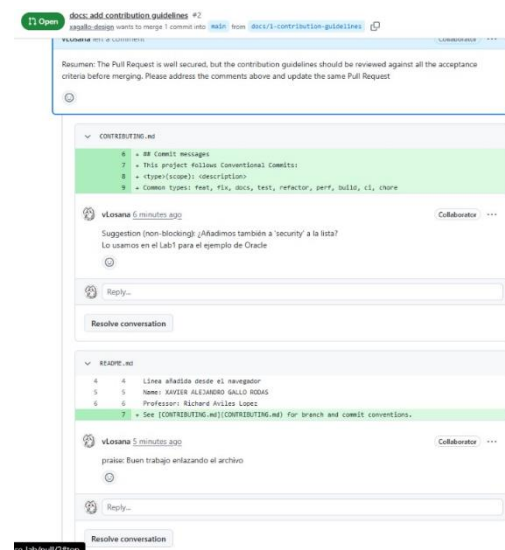
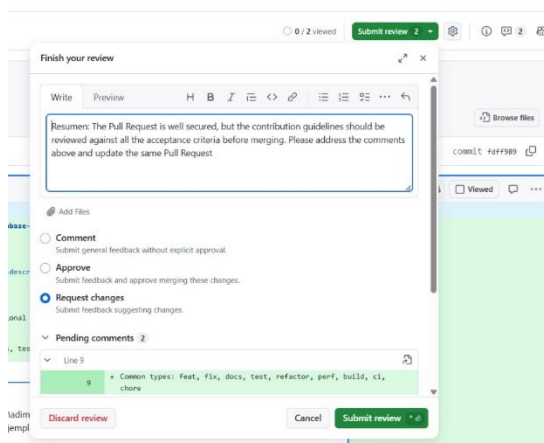
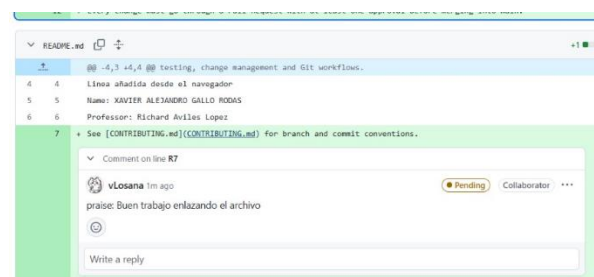
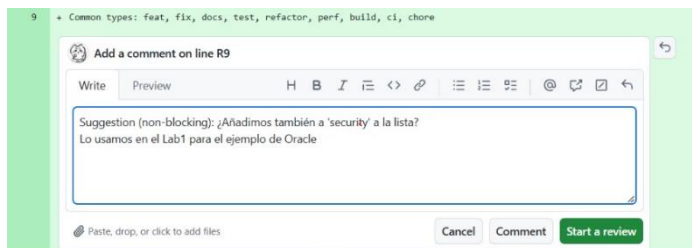
Podemos ver que:



PARTE F: Code Review - Solicitud de cambios (Reviewer)



El reviewer revisa los cambios, pone comentarios y envía el review:



PARTE G: Responder a la revisión (Autor)

El autor revisa los comentarios del review:

The screenshot shows a GitHub pull request interface. At the top, a notification states "vLosana requested changes 3 minutes ago". Below this, a comment from vLosana is displayed: "Resumen: The Pull Request is well secured, but the contribution guidelines should be reviewed against all the acceptance criteria before merging. Please address the comments above and update the same Pull Request".

The review focuses on the `CONTRIBUTING.md` file. The file content is shown with line numbers 6 through 9. A comment from vLosana, posted 9 minutes ago, suggests adding 'security' to the list of common types: "Suggestion (non-blocking): ¿Añadimos también a 'security' a la lista? Lo usamos en el Lab1 para el ejemplo de Oracle".

The author's response is shown below the comment. It includes a "Reply..." input field and a "Resolve conversation" button. The response text is: "praise: Buen trabajo enlazando el archivo".

Below the response, the `README.md` file is shown with line numbers 4 through 7. The content includes a new line added from the navigator: "Linea añadida desde el navegador". The text also includes the author's name and professor, and a reference to the `CONTRIBUTING.md` file for branch and commit conventions.

Another comment from vLosana, posted 7 minutes ago, is shown below the `README.md` file. It includes a "praise: Buen trabajo enlazando el archivo" and a "Resolve conversation" button.

Modificamos el archivo:

```

1 CONTRIBUTING.md > # Contributing to oracle-database-lab > ## Commit messages
2 # Contributing to oracle-database-lab
3
4 ## Branch naming
5 <type>/<issue number>-<short-description>, e.g. feature/56-customer-search, fix/85-customer-email-validation
6
7 ## Commit messages
8 This project follows Conventional Commits:
9 <type>(<scope>): <description>
10 Common types: feat, fix, docs, test, refactor, perf, build, ci, chore, security
11
12 ## Pull Requests
13 Every change must go through a Pull Request with at least one approval before merging into main.

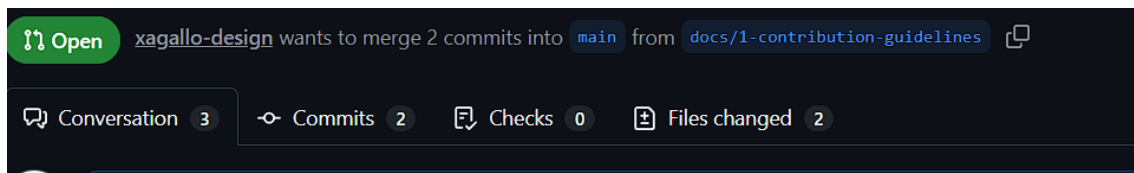
```

Subimos los cambios:

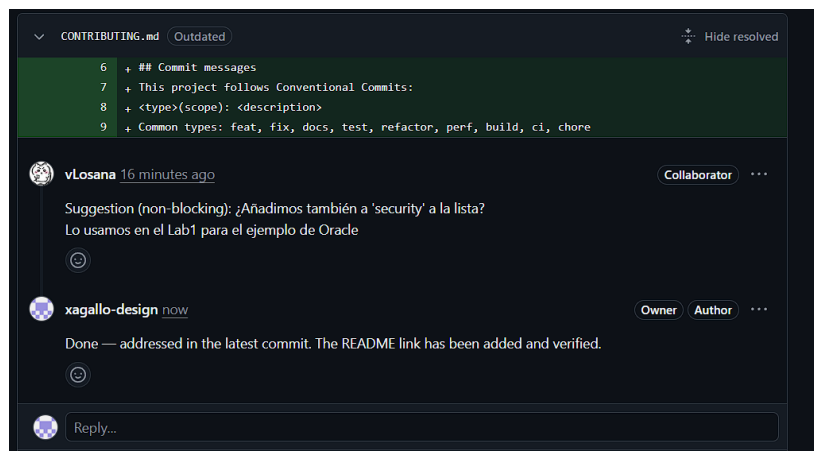
```
xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$ git add CONTRIBUTING.md
$ git commit -m "docs: add security to conventional commit types list"
warning: in the working copy of 'CONTRIBUTING.md', LF will be replaced by CRLF the next time Git touches it
[docs/1-contribution-guidelines 9e3bbb4] docs: add security to conventional commit types list
1 file changed, 1 insertion(+), 1 deletion(-)

xavi2@Xavi MINGW64 ~/oracle-database-lab (docs/1-contribution-guidelines)
$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/xagallo-design/oracle-database-lab.git
fdff989..9e3bbb4 docs/1-contribution-guidelines -> docs/1-contribution-guidelines
```

Se pueden ver los cambios



Resolvemos la conversación:



Segunda revisión:

Ahora el revisor vuelve a entrar al mismo pull request. Observará 2 commits, file changed 2, y aparecerá en la parte superior derecha « Changes requeste ». Un Pull Request puede contener varios commits.

docs: add contribution guidelines #2

Changes requested Code

Open xagallo-design wants to merge 2 commits into main from docs/1-contribution-guidelines

Conversation 4 Commits 2 Checks 0 Files changed 2 +13

Commits on Sep 25, 2026

- docs: add contribution guidelines fdf989 Xavo06 committed 33 minutes ago
- docs: add security to conventional commit types list 9e3bbb4 Xavo06 committed 5 minutes ago

Merge status Preview X

Changes requested
1 change requested by reviewers with write access.

1 requested change

docs: add security to conventional commit types list

Xavo06 committed 8 minutes ago commit 9e3bbb4

CONTRIBUTING.md

```
@@ -6,7 +6,7 @@
## Commit messages
This project follows Conventional Commits:
<type>(scope): <description>
- Common types: feat, fix, docs, test, refactor, perf, build, ci, chore
+ Common types: feat, fix, docs, test, refactor, perf, build, ci, chore, security
```

Comment on line R9

vlosana 1m ago Pending Collaborator

approve: cambio sugerido añadido correctamente

Write a reply

Pull Requests

Every change must go through a Pull Request with at least one approval before merging into main.

Finish your review

Write Preview

Approve: cambios aprobados

Paste, drop, or click to add files

☐ Comment
Submit general feedback without explicit approval.

☒ Approve
Submit feedback and approve merging these changes.

☐ Request changes
Submit feedback suggesting changes.

Pending comments 1

Line 9

+ Common types: feat, fix, docs, test, refactor, perf, build, ci, chore, security

approve: cambio sugerido añadido correctamente

Discard review Cancel Submit review

Changes approved
1 approving review by reviewers with write access.

1 approval

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line. View command line instructions

Still in progress? Convert to draft

Add a comment

PARTE H Comprobar que la protección funciona

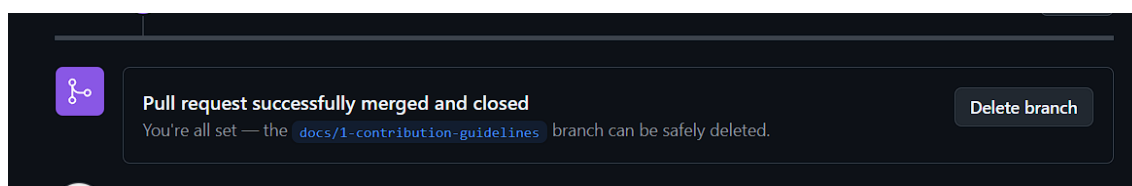
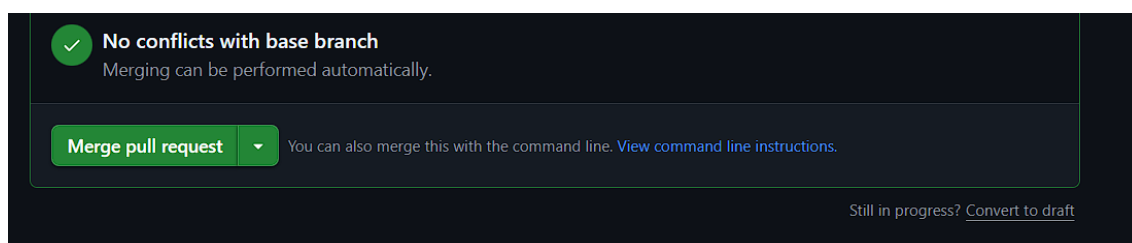
Antes de fusionar, hagamos una prueba deliberada: intenta hacer push directo a main desde la terminal:

Da el siguiente error: remote: error: GH006: Protected branch update failed for refs/heads/main.

Este rechazo es exactamente lo que buscábamos en la Parte B. Deshago el experimento:

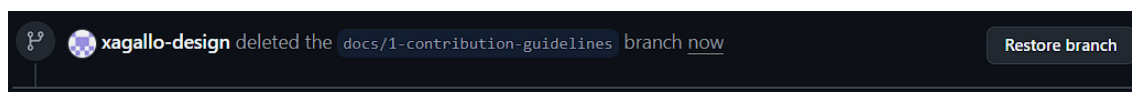
PARTE I Fusionar el pull request:

Con la aprobación obtenida del Revisor, GitHub habilita el botón Merge pull request. Dependiendo de los permisos, para este laboratorio, el autor hará la fusión.



PARTE J Limpieza posterior al merge:

Borramos la rama:



Sincronizamos y limpiamos en la terminal:

```
xavi12@Xavi1 MINGW64 ~/oracle-database-lab (main)
$ echo "test" >> README.md
git add README.md
git commit -m "chore: direct change on main"
git push --
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 1.50 KiB | 110.00 KiB/s, done.
From https://github.com/xagallo-design/oracle-database-lab
  7ca0849..46c03a1  main      -> origin/main
Updating 447e99c..46c03a1
Fast-forward
 CONTRIBUTING.md | 12 ++++++++
 README.md       |  5 +++-
 2 files changed, 15 insertions(+), 2 deletions(-)
 create mode 100644 CONTRIBUTING.md
warning: deleting branch 'docs/1-contribution-guidelines' that has been merged to
'refs/remotes/origin/docs/1-contribution-guidelines', but not yet merged to HEAD
Deleted branch docs/1-contribution-guidelines (was 9e3bbb4).
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

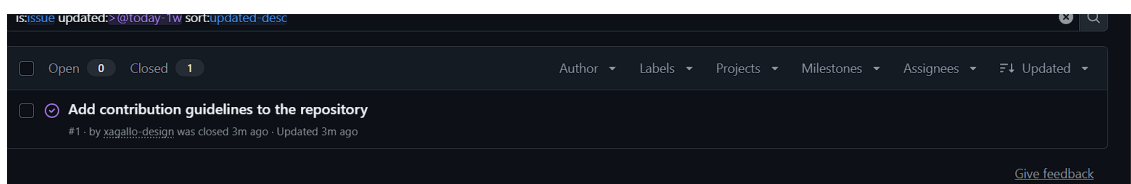
Comprobamos la integración final:

```
## Branch naming
<type>/<issue number>-<short-description>, e.g. feature/56-customer-search, fix/85-customer-email-validation

## Commit messages
This project follows Conventional Commits:
<type>(scope): <description>
Common types: feat, fix, docs, test, refactor, perf, build, ci, chore, security

## Pull Requests
Every change must go through a Pull Request with at least one approval before merging into main.
```

Comprobamos el cierre del issue



3. PREGUNTAS DE LA PRÁCTICA:

1. ¿Por qué un Issue sin criterios de aceptación es un problema, aunque la descripción general parezca clara?

Porque genera ambigüedad sobre la definición de "terminado" (Definition of Done). Sin una lista de comprobación verificable, ni el autor sabe exactamente cuándo ha completado el alcance del trabajo, ni el revisor cuenta con un criterio objetivo e inequívoco para validar si el Pull Request cumple con todos los requisitos solicitados.

2. Explica la diferencia entre "Refs #N" y "Closes #N" en un mensaje de commit o en la descripción de un Pull Request.

- Refs #N: Crea una referencia o enlace cruzado visible en el historial del Issue #N, pero mantiene el Issue abierto. Se utiliza durante el desarrollo para registrar avances intermedios.
- Closes #N: Es una palabra clave reservada de GitHub que cierra automáticamente el Issue #N en el momento exacto en que la rama o el Pull Request se fusiona (merge) dentro de la rama principal (main).

3. ¿Qué ocurre exactamente si intentas hacer git push directamente sobre una branch main protegida? ¿Es un error tuyo o un fallo del sistema?

GitHub rechaza la subida mostrando un error en la consola (por ejemplo, remote: error: GH006: Protected branch update failed y Changes must be made through a pull request).

No es un fallo del sistema, sino el comportamiento previsto de la regla de protección (Branch Protection Rule), la cual bloquea deliberadamente los envíos directos para forzar a que todo cambio se integre exclusivamente mediante un Pull Request con revisiones y aprobaciones previas.

4. Un compañero te dice: "he aprobado el PR sin mirar los archivos, total ya me fío". ¿Qué riesgo tiene esa forma de revisar?

Esa práctica (conocida como rubber-stamping) elimina por completo la barrera de control de calidad del repositorio. Aumenta el riesgo de introducir inadvertidamente bugs en producción, archivos innecesarios o accidentales, vulnerabilidades de seguridad, regresiones o inconsistencias de formato y arquitectura que la persona que programó el código pudo haber pasado por alto.

5. Si el reviewer pide un cambio y tú ya habías hecho push de tu branch, ¿tienes que abrir un Pull Request nuevo? Explica qué ocurre técnicamente con el PR existente cuando haces un nuevo commit.

No se debe abrir un PR nuevo.

Técnicamente, un Pull Request está vinculado a la rama origen (head branch). Al realizar nuevos commits locales y hacer git push a esa misma rama remota, GitHub detecta automáticamente los nuevos hashes de commit y actualiza el historial y la pestaña Files changed del PR existente, preservando todo el contexto y la discusión previa.

6. Describe con tus palabras la diferencia entre Merge commit, Squash and merge y Rebase and merge. ¿Cuál usarías para una branch con commits "wip", "fix", "fix2", "ok ya"?

- Merge commit: Preserva todos los commits individuales de la rama y crea un commit adicional de unión con dos padres, manteniendo el historial detallado de bifurcación.
- Squash and merge: Condensa todos los commits de la rama secundaria en un único commit limpio y atómico que se aplica sobre main.
- Rebase and merge: Reaplica los commits de la rama uno por uno sobre la punta de main de manera lineal, sin generar un commit de fusión.
- Elección: Para commits sucios o de trabajo como "wip", "fix", etc., se debe usar Squash and merge para no ensuciar el historial de main con pasos intermedios irrelevantes.

7. ¿Por qué borrar una branch después del merge no elimina el trabajo realizado en ella?

Porque en Git las ramas son simplemente punteros móviles ligeros que hacen referencia a un hash de commit específico. Una vez que el cambio fue fusionado, los objetos del commit y los cambios del árbol ya forman parte del historial de main, por lo que eliminar la rama solo destruye el puntero temporal de trabajo, no los datos integrados.

8. ¿Qué información debería contener siempre la descripción de un Pull Request, como mínimo?

Debe responder a cuatro puntos clave mediante una plantilla estructurada:

- Summary: Qué hace el cambio de forma resumida.

- Changes: Lista detallada de los archivos o componentes modificados.
- Testing: Cómo se ha verificado el cambio o qué pruebas se realizaron.
- Related Issue: El enlace al ticket que resuelve (por ejemplo, Closes #1).

9. Un reviewer escribe solo "esto está mal" como comentario. ¿Qué le falta a ese comentario para ser útil? Reescríbelo tú con un ejemplo inventado.

Le falta empatía cognitiva, contexto y accionabilidad: no explica qué se observa, por qué supone un problema ni cómo se propone solucionarlo, generando una actitud defensiva. Reescritura (usando Conventional Comments):

issue (blocking): Este método no valida si la conexión a la base de datos es nula antes de ejecutar la consulta, lo que provocará un NullPointerException si el pool falla. Sugiero envolver la llamada en un bloque condicional o usar un Optional.

10. ¿Qué diferencia hay entre que main esté protegida y que simplemente el equipo "se ponga de acuerdo" en no hacer push directo?

"Ponerse de acuerdo" es una política organizativa dependiente exclusivamente de la disciplina humana, vulnerable a errores involuntarios, prisas o descuidos.

Tener main protegida convierte esa política en un control técnico automatizado y forzado por la plataforma, donde GitHub rechaza a nivel de servidor cualquier intento de eludir el flujo establecido (PR y revisiones).

11. Explica con un ejemplo propio la diferencia entre un comentario issue: (blocking) y uno nitpick: (if-minor) en formato Conventional Comments.

- issue (blocking): Representa un error o problema crítico que impide aprobar y fusionar el PR hasta que sea solucionado.
Ejemplo: issue (blocking): La credencial de conexión está hardcodeada en el archivo; debe extraerse como variable de entorno antes del merge.
- nitpick: (if-minor): Representa un detalle menor de estilo, ortografía o formato que no bloquea la integración y cuya resolución queda a criterio del autor.
Ejemplo: nitpick: (if-minor): Falta un espacio en blanco entre el operador de asignación y la variable en la línea 42 para seguir el estándar del proyecto.

12. Si tu próximo commit es feat!: cambia la firma de la función principal de la API, ¿qué tipo de versión SemVer se dispara y por qué?

Se dispara un incremento de versión MAJOR (por ejemplo, de 2.1.0 a 3.0.0).

Ocurre porque el signo de exclamación ! junto al tipo (feat!:) denota un BREAKING CHANGE, lo que significa que introduce cambios que rompen la compatibilidad hacia atrás con las versiones anteriores de la API.

13. ¿Por qué abrir un Draft PR desde el primer commit estructural puede ahorrar tiempo al equipo, aunque parezca más lento para el autor individual?

Porque aplica el principio de Fail Fast (fallar rápido). Permite discutir y validar la arquitectura y el enfoque general con revisores o miembros sénior antes de invertir

días programando lógica detallada o escribiendo suites completas de pruebas sobre un diseño erróneo, evitando así la falacia del coste hundido.